

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Projeto e Seminário 2025/2026

Organização do Projeto

Diário LX

Plataforma de Jornalismo Digital

Estudantes: Jessé Alencar, n.º 51745, a51745@alunos.isel.pt
Julianne Lobato, n.º 51191, a51191@alunos.isel.pt

Orientadores: Artur Ferreira, artur.ferreira@isel.pt — ISEL
Filipe Freitas, filipe.freitas@isel.pt — ISEL
Fátima Cardoso, mlcardoso@escs.ipl.pt — ESCS

Julho de 2026

1 Repositório do Projeto

Toda a implementação e documentação do projeto **Diário LX** estão disponíveis publicamente no *GitHub*:

<https://github.com/jfmalencar/DiarioLX>

2 Organização do Projeto

O projeto está organizado num único repositório. O código fonte encontra-se na pasta `code/`, dividida em dois diretórios principais: o *backend* (JVM/Kotlin) e o *frontend* (React/TypeScript).

2.1 *Backend* — `code/jvm/`

```
code/jvm/
|-- modules/
|   |-- domain/           % Entidades e regras de negocio
|   |-- http/             % Controllers, DTOs e autenticacao
|   |-- services/         % Logica de negocio e validacao
|   |-- repository/       % Interfaces de acesso a dados
|   |-- repository-jdbi/  % Implementacoes JDBC + SQL
|   |-- storage/          % Interface de armazenamento
|   |-- storage-s3/       % Implementacao S3 / SeaweedFS
|   '-- host/             % Ponto de entrada e configuracao
|-- build.gradle.kts
|-- settings.gradle.kts
|-- seaweed-config.json
'-- docker-compose.yml
```

2.2 *Frontend* — `code/js/`

```
code/js/
|-- src/
|   |-- app/
|   |   '-- providers/    % Bootstrap, autenticacao e i18n
|   |-- assets/          % Logotipo, icones e traducoes
|   |-- config/          % Variaveis de ambiente
|   |-- layouts/         % Layouts partilhados
|   |-- modules/
|   |   |-- backoffice/  % Interface de gestao editorial
|   |   '-- public/      % Interface publica do jornal
```

```
| | -- shared/
| | | -- components/    % Componentes reutilizaveis
| | | -- hooks/         % Hooks de acesso ao servicos
| | | -- services/      % Interfaces e implementacoes
| | | -- types/         % Tipos e interfaces partilhados
| | ' -- utils/         % Funcoes utilitarias
| ' -- tests/           % Testes Playwright
|-- index.html
|-- index.tsx
' -- package.json
```

3 Implantação e Utilização

3.1 Pré-requisitos

- **Docker** e **Docker Compose** instalados.
- **Java 17+** instalado (necessário para executar o *Gradle Wrapper*).
- Acesso à linha de comandos (terminal).

3.2 Modos de Execução

O projecto suporta dois modos de execução distintos:

Desenvolvimento local O *backend* é executado directamente pelo *IntelliJ IDEA* e o *frontend* pelo servidor de desenvolvimento *Vite* (`npm run dev`). As variáveis de ambiente são carregadas a partir do ficheiro `.env` na raiz de `code/jvm/`. Apenas a base de dados e o *SeaweedFS* são executados via *Docker Compose*:

```
cd code/jvm
docker compose up -d
```

Staging Todos os serviços — *backend*, base de dados, *SeaweedFS* e a imagem *web* (*frontend* servido por *Nginx*, que actua também como *reverse proxy*) — são executados em *containers Docker*. As variáveis de ambiente são carregadas a partir do ficheiro `.env.staging`. Este é o modo descrito nas secções seguintes, usando as *tasks* Gradle `buildImageAll` e `allUp`.

3.3 Configuração Inicial

Antes de executar qualquer comando, é necessário colocar dois ficheiros de configuração na pasta `code/jvm/`:

3.3.1 Ficheiro *.env.staging*

Criar o ficheiro `code/jvm/.env.staging` com o seguinte conteúdo:

```
PORT=8180

APP_PUBLIC_URL=http://localhost:8180

S3_PUBLIC_ENDPOINT=http://localhost:${PORT}
S3_ENDPOINT=http://diariolx-seaweed-s3:8333
S3_REGION=us-east-1
S3_ACCESS_KEY=DEV-ACCESS-KEY
S3_SECRET_KEY=DEV-SECRET-KEY
S3_BUCKET=media
S3_PATH_STYLE_ACCESS=true

SCALAR_PATH=/docs

JWT_SECRET=QBqCNu2Dusr161yq7foRF2U6MLjKSG4oG8j7T7RNbk4=
JWT_ACCESS_EXPIRATION_MS=600000
JWT_REFRESH_EXPIRATION_MS=604800000

POSTGRES_USER=dbuser
POSTGRES_PASSWORD=changeit
POSTGRES_DB=db
POSTGRES_HOST=diariolx-postgres-test
POSTGRES_PORT=5432

DB_URL=jdbc:postgresql://${POSTGRES_HOST}:${POSTGRES_PORT}/${
    POSTGRES_DB}\
?user=${POSTGRES_USER}&password=${POSTGRES_PASSWORD}
```

3.3.2 Ficheiro *seaweed-config.json*

Alterar o ficheiro `code/jvm/seaweed-config.json` com o seguinte conteúdo:

```
{
  "identities": [
    {
      "name": "anonymous",
      "actions": ["Read"],
      "resources": ["buckets/media/*"]
    },
  ],
```

```
{
  "name": "dev-user",
  "credentials": [
    {
      "accessKey": "DEV-ACCESS-KEY",
      "secretKey": "DEV-SECRET-KEY"
    }
  ],
  "actions": ["Read", "Write", "List", "Tagging", "Admin"],
  "resources": ["buckets/media/*"]
}
```

Este ficheiro configura as permissões do *SeaweedFS* — o sistema de armazenamento de ficheiros multimédia. O utilizador `anonymous` tem permissão de leitura pública, enquanto o `dev-user` tem acesso total para operações do *backend*.

3.4 Construção e Arranque

Com os ficheiros de configuração em lugar, navegar para a pasta `code/jvm/` e executar os seguintes comandos pela ordem indicada.

3.4.1 Passo 1 — Construir as imagens Docker

```
cd code/jvm
./gradlew buildImageAll
```

Este comando compila o projecto e constrói todas as imagens *Docker* necessárias (*backend*, *frontend* e infraestrutura).

3.4.2 Passo 2 — Iniciar todos os serviços

```
./gradlew allUp
```

Este comando inicia todos os serviços orquestrados via *Docker Compose*. Após a conclusão, a aplicação estará disponível na porta 8180.

3.5 Resumo dos *Endpoints*

Após o arranque, todos os serviços são acessíveis através de uma única porta:

<i>Endpoint</i>	Descrição	URL
<i>Frontend</i>	Interface web (site público e <i>backoffice</i>)	<code>localhost:8180</code>
API REST	<i>Backend</i> Spring Boot	<code>localhost:8180/api</code>
Documentação	Scalar (especificação <i>OpenAPI</i>)	<code>localhost:8180/docs</code>
<i>Media</i>	Ficheiros multimédia (<i>SeaweedFS</i>)	<code>localhost:8180/media/...</code>

Nota: O *Nginx* actua como *reverse proxy*, encaminhando todos os pedidos através da porta 8180. Não é necessário expor portas individuais de cada serviço.

3.6 Convite Inicial de Administrador

Na primeira execução, é inserido automaticamente na base de dados um convite de administrador com o código:

```
SUPER - ADMIN - INVITE
```

Aceder a `localhost:8180/backoffice/register` e usar este código para criar uma conta com o papel de ADMIN.

4 Tecnologias Utilizadas

Tecnologia	Utilização
Kotlin + Spring Boot	<i>Backend</i> e API REST
JDBI	Acesso à base de dados
PostgreSQL	Persistência de dados estruturados
SeaweedFS (S3)	Armazenamento de ficheiros multimédia
React + TypeScript	Interface <i>web</i> (<i>backoffice</i> e site público)
Vite	<i>Bundler</i> e servidor de desenvolvimento
Playwright	Testes <i>end-to-end</i> do <i>frontend</i>
Docker + Compose	<i>Containerização</i> e orquestração de serviços
Nginx	<i>Reverse proxy</i> (implantação)